

Problem Set 4, Part I

Please start your answer to each problem on a new page, as we have done below!

Problem 1: Understanding and using inheritance

1-1) Automobile and Vehicle

1-2) Yes. `t.getMileage()` compiles because `Taxi` → `Automobile` → `Vehicle`, and `getMileage()` is part of the shared vehicle behavior. Assuming `getMileage()` is public/protected in `Vehicle`, `Taxi` inherits it.

1-3) Add your definition of the `MovingVan` class below:

Java

```
public class MovingVan extends Truck {
    private final int widthIn;    // inches, ≥ 0
    private final int heightIn;   // inches, ≥ 0
    private final int depthIn;   // inches, ≥ 0
    private final boolean hasLock;

    public MovingVan(String make, String model, int year, int
wheels,
                    int width, int height, int depth, boolean
hasLock) {
        super(make, model, year, wheels);
        if (wheels < 0) throw new
IllegalArgumentException("wheels < 0");
        if (width < 0 || height < 0 || depth < 0)
            throw new IllegalArgumentException("dimensions < 0");
        this.widthIn = width;
        this.heightIn = height;
        this.depthIn = depth;
        this.hasLock = hasLock;
    }

    public int getWidthIn() { return widthIn; }
```

```
public int getHeightIn() { return heightIn; }
public int getDepthIn() { return depthIn; }
public boolean hasLock() { return hasLock; }

public String toString() {
    String s = getMake()+" " +getModel()+", dimensions =
"+widthIn+ " x " +heightIn+ " x " +depthIn+", " +getNumAxles();
    return s;
}
}
```

Problem 2: Inheritance and polymorphism

2-1)

Foo overrides equals() from java.lang.Object.

2-2)

Fields in a Hoo object (declared + inherited):

- From Foo
 - a:int,
 - b:String
- From Hoo
 - t:int,
 - u:int,
 - v:double.

2-3)

which println statement?	which method is called?	will the call compile (yes/no?)	if the call compiles, which version of the method will be called?
first one	one()	yes	the Moo version
second one	two()	yes	Goo's two (Moo inherits Goo's override)
third one	three(double)	no	– (method not in Moo nor its superclasses)
fourth one	equals(Object)	yes	Foo's equals (Moo→Goo→Foo; Moo doesn't override)
fifth one	toString()	yes	Goo's toString (Moo inherits Goo's override)

2-4)

Java

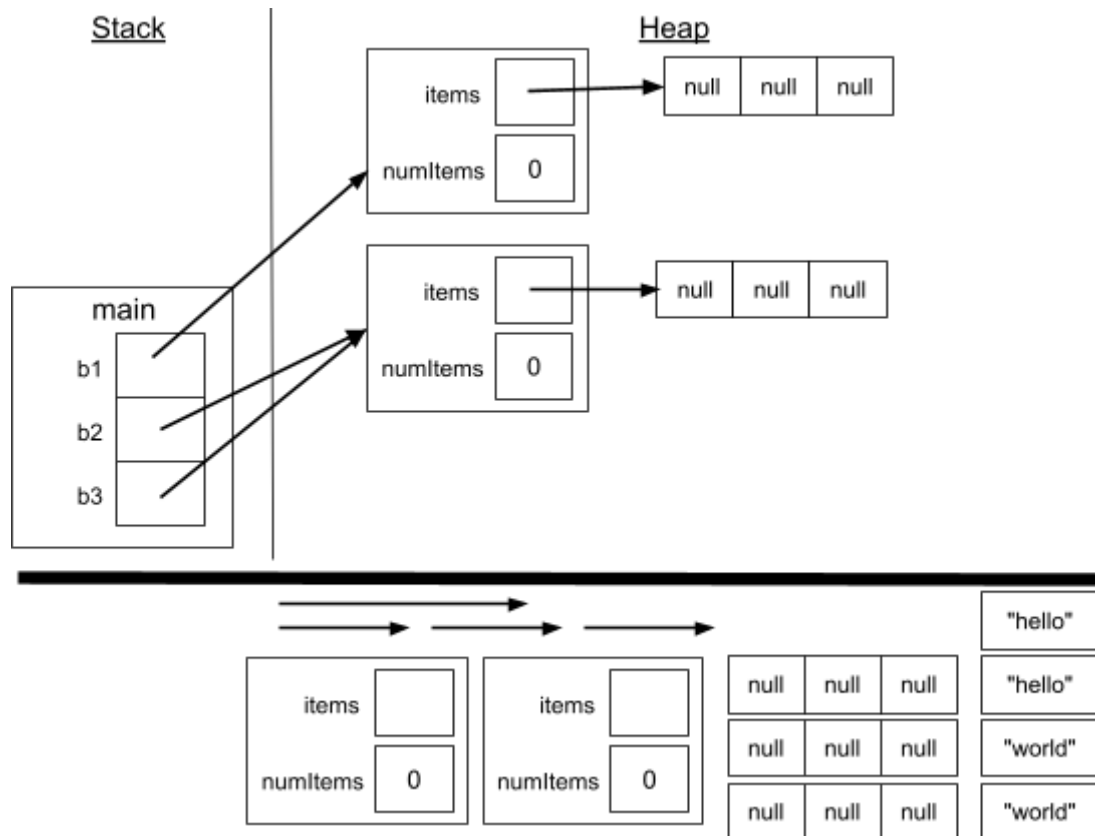
```
public double avg() {  
  
    double sum = getA() + getT() + getU() + getV();  
  
    return sum / 4.0;  
  
}
```

2-5)

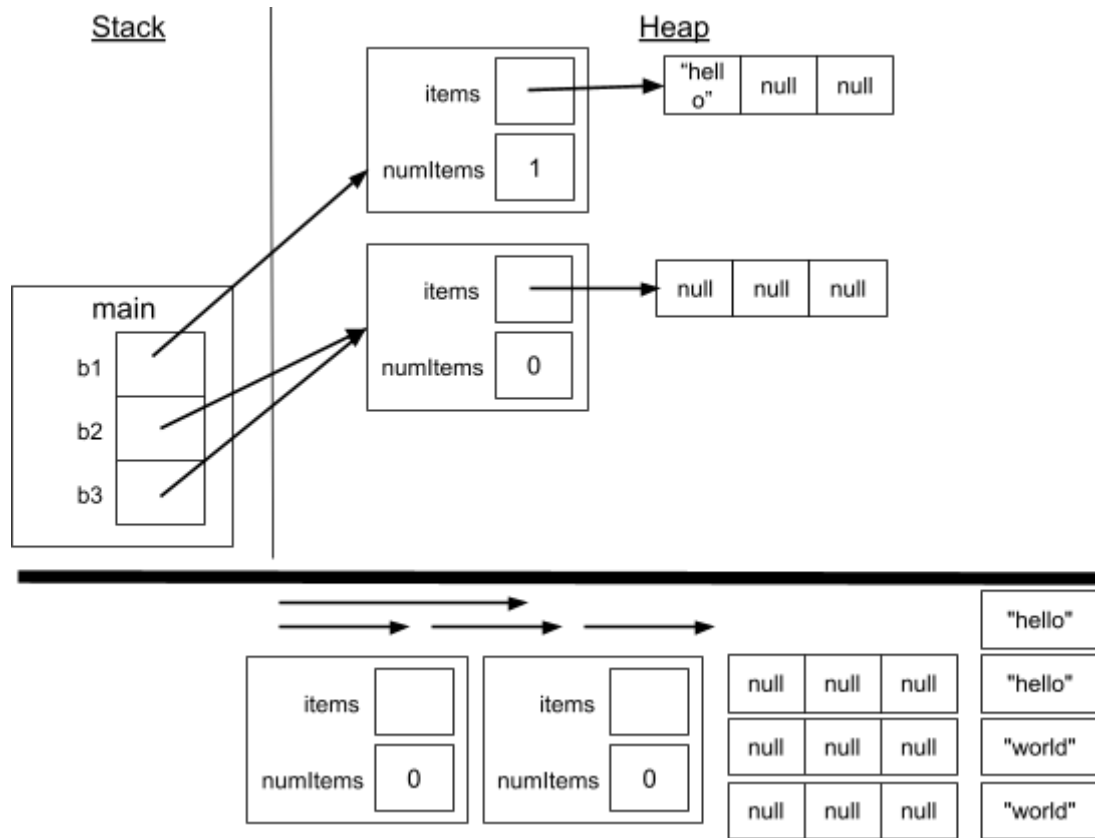
- a)** Foo z = new Moo(); Allowed. Upcasting a subclass (Moo) to superclass (Foo).
- (b)** Hoo t = new Foo(); Not allowed. Cannot assign a superclass instance to subclass reference.
- (c)** Goo w = new Hoo(); Not allowed. Hoo doesn't extend Goo.
- (d)** Foo y = new Goo(); Allowed. Upcasting Goo → Foo.

Problem 3: Memory management and the ArrayBag class

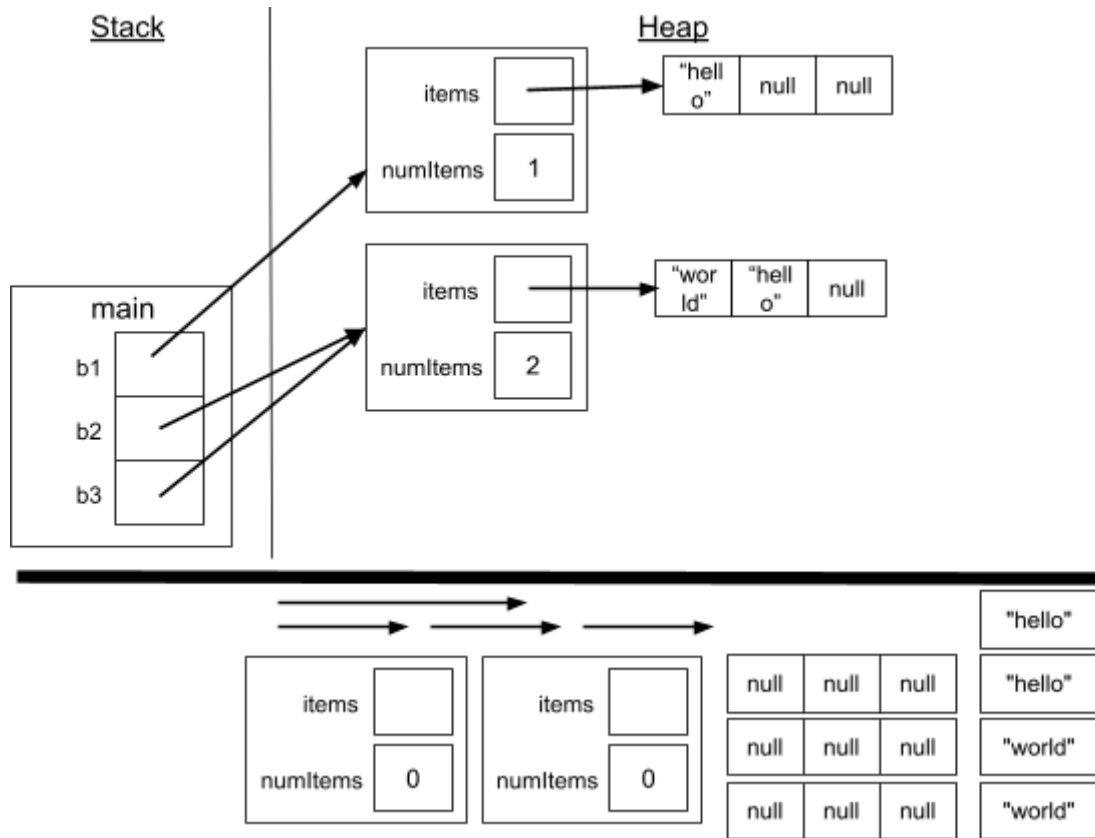
3-1)



3-2)



3-3)



3-4)

Whatever exact toString() format the provided ArrayBag uses, the contents are:

- `b1` → a bag with just hello
- `b2` → a bag with world and hello
- `b3` → identical to `b2` (same object)